

# Coalesced memory access

GPU Programming – M2 DiPaC, Université Paris-Saclay – Session 3

Oguz Kaya

Assistant Professor

Université Paris-Saclay and LISN, Gif-sur-Yvette, France

# Objectives

- Understand how the 32 threads of a warp access global memory *together*
- Recognise **coalesced** and **strided** access, and know what each one costs
- Answer last session's question: which of the four 2D-array kernels is fastest, and why

# Outline

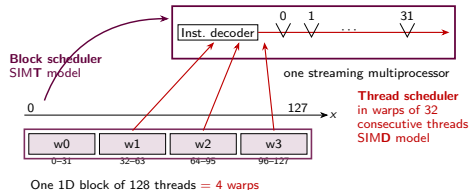
- 1 Recap of session 2
- 2 Coalesced memory access
- 3 The kernels of session 2: coalesced or not?
- 4 Rules

- 1 Recap of session 2
- 2 Coalesced memory access
- 3 The kernels of session 2: coalesced or not?
- 4 Rules

# Recap: warps, the unit of execution

Make blocks out of a whole number of **warps**

- One instruction decoder drives 32 CUDA cores, i.e. 32 threads at once
- Each group of 32 consecutive threads of a block is a **warp**: the unit the hardware actually schedules
- The threads of a warp are **synchronous**: they execute the same instruction at the same time, each on its own data
- The SM's scheduler picks a warp of a resident block and issues its next instruction to the 32 cores
- A block of 100 threads is 4 warps, the last one 7/8 empty (4 of 32 threads used) – hence “a multiple of 32”

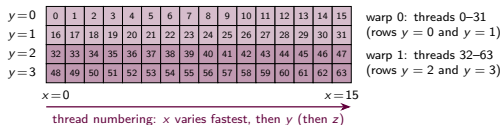


# Recap: how the hardware numbers the threads of a block

A 2D or 3D block (dim3 dimBlock) is still a **sequence** of threads for the hardware.

- Threads are ordered with **threadIdx.x increasing first**, then threadIdx.y, then threadIdx.z
- That sequence is cut into **warps of 32**: for  $\text{blockDim.x} \geq 32$ , a warp is 32 threads with consecutive threadIdx.x and the same y, z
- When  $\text{blockDim.x} < 32$ , several consecutive threadIdx.y complete the warp (then .z):  $16 \times 4$  block, two rows per warp
- A fact about the hardware, not a choice of the programmer
- **The fact of the day**: it decides which of last session's kernels are fast

block  $16 \times 4$  (blockDim.x=16, blockDim.y=4): 64 threads, two warps



block  $32 \times 2$ : two warps, one per row



# Recap: row-major storage

Last session: C stores  $A[N][N]$  **row after row**. Today the arrays are 1D arrays of  $N^2$  floats, and we choose the interpretation:

**row-major:**  $A[i][j]$  is  $A[i \cdot N + j]$

- Consecutive  $j$  (same row) are 4 bytes apart: **contiguous**
- Consecutive  $i$  (same column) are  $4N$  bytes apart: **strided**

$A$  with  $N = 4$ , as we think of it

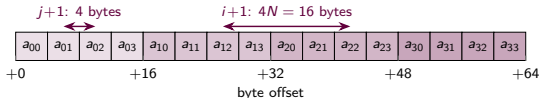
|         |          |          |          |          |
|---------|----------|----------|----------|----------|
| $i = 0$ | $a_{00}$ | $a_{01}$ | $a_{02}$ | $a_{03}$ |
| $i = 1$ | $a_{10}$ | $a_{11}$ | $a_{12}$ | $a_{13}$ |
| $i = 2$ | $a_{20}$ | $a_{21}$ | $a_{22}$ | $a_{23}$ |
| $i = 3$ | $a_{30}$ | $a_{31}$ | $a_{32}$ | $a_{33}$ |

row  $i$  starts at byte  $i \cdot 4N$

element  $j$  of it is  $4j$  bytes further

so float  $A[N][N]$  and float  $A[N \times N]$   
(indexed  $A[i \cdot N + j]$ ) are the same bytes

$j =$  the same bytes in memory: one row after another



# Recap: column-major storage – the mirror image

The other interpretation of the same  $N^2$  floats: **column after column** (the Fortran/BLAS convention)

**column-major:**  $A[i][j]$  is  $A[i + j \cdot N]$

- Consecutive  $i$  (same column) are 4 bytes apart:  
**contiguous**
- Consecutive  $j$  (same row) are  $4N$  bytes apart: **strided**
- Same bytes, same kernels, one index formula changed: every kernel of the lab comes in **both** versions

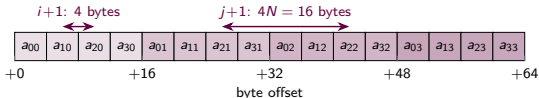
$A$  with  $N = 4$ , as we think of it

|         |          |          |          |          |
|---------|----------|----------|----------|----------|
| $i = 0$ | $a_{00}$ | $a_{01}$ | $a_{02}$ | $a_{03}$ |
| $i = 1$ | $a_{10}$ | $a_{11}$ | $a_{12}$ | $a_{13}$ |
| $i = 2$ | $a_{20}$ | $a_{21}$ | $a_{22}$ | $a_{23}$ |
| $i = 3$ | $a_{30}$ | $a_{31}$ | $a_{32}$ | $a_{33}$ |

column  $j$  starts at byte  $j \cdot 4N$   
element  $i$  of it is  $4i$  bytes further

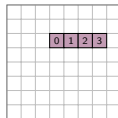
no C array type does this for us:  
`float A[N*N]` indexed `A[i + j*N]`

$j =$  the <sup>0</sup> <sup>1</sup> <sup>2</sup> <sup>3</sup> same bytes in memory: one column after another

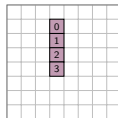


# Recap: what a block computes – the cases of session 2

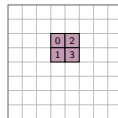
One thread per element of  $A[N][N]$ , blockSize threads per block; which elements does a block get? (figure: blockSize = 4)



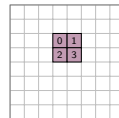
(a) 1D block:  
a piece of a **row**



(b) 1D block:  
a piece of a **column**



(c) block =  $2 \times 2$  **tile**,  
threads column aligned



(d) block =  $2 \times 2$  **tile**,  
threads row aligned

- Before these: one thread per *row* (too few threads), and one thread per element with **blocks of one thread** (warps 1/32 full)
- (a), (b): a 1D block is a piece of a row, or a piece of a column
- (c), (d): a block is a square **tile**, threads column aligned or row aligned – each with an *alternative* numbering of the blocks
- All correct; all cover every element once. The question left open: **which one is fastest?**

# Recap: from indices to $(i, j)$

Every case is the same formula, with a block of  $\text{side}_i \times \text{side}_j$  elements:

$$i = b_i \cdot \text{side}_j + t_i, \quad j = b_j \cdot \text{side}_i + t_j$$

| case     | $\text{side}_i, \text{side}_j$ | 1D indices: $(b_i, b_j)$                           | 1D indices: $(t_i, t_j)$                             | 2D: $(b_i, b_j)$ | 2D: $(t_i, t_j)$ |
|----------|--------------------------------|----------------------------------------------------|------------------------------------------------------|------------------|------------------|
| (a)      | (1, BS)                        | $\text{bx}/\text{bpr}, \text{bx} \bmod \text{bpr}$ | 0, tx                                                | bx, by           | 0, tx            |
| (b)      | (BS, 1)                        | $\text{bx} \bmod \text{bpc}, \text{bx}/\text{bpc}$ | tx, 0                                                | bx, by           | tx, 0            |
| (c)      | (side, side)                   | $\text{bx}/\text{tpr}, \text{bx} \bmod \text{tpr}$ | $\text{tx} \bmod \text{side}, \text{tx}/\text{side}$ | bx, by           | tx, ty           |
| (d)      | (side, side)                   | $\text{bx}/\text{tpr}, \text{bx} \bmod \text{tpr}$ | $\text{tx}/\text{side}, \text{tx} \bmod \text{side}$ | bx, by           | ty, tx           |
| (c) alt. | (side, side)                   | $\text{bx} \bmod \text{tpc}, \text{bx}/\text{tpc}$ | as (c)                                               | by, bx           | as (c)           |
| (d) alt. | (side, side)                   | $\text{bx} \bmod \text{tpc}, \text{bx}/\text{tpc}$ | as (d)                                               | by, bx           | as (d)           |

- $\text{bx}, \text{by} = \text{blockIdx.x}, \text{.y}; \quad \text{tx}, \text{ty} = \text{threadIdx.x}, \text{.y}$
- $\text{BS} = \text{blockSize} = 1024; \quad \text{side} = \sqrt{\text{BS}} = 32$
- $\text{bpr}, \text{bpc} = \lceil N/\text{BS} \rceil$  blocks per row / per column;  $\text{tpr}, \text{tpc} = \lceil N/\text{side} \rceil$  tiles per row / per column
- (a), (b): a 1D block is a  $1 \times \text{BS}$  or  $\text{BS} \times 1$  tile – one of the two thread coordinates is always 0
- With 2D indices, `blockIdx` and `threadIdx` simply **replace**  $(b_i, b_j)$  and  $(t_i, t_j)$ ; the mappings are the same
- Today: **what 32 consecutive threadIdx.x touch** – (a), (d): a row piece; (b), (c): a column piece

- 1 Recap of session 2
- 2 Coalesced memory access
- 3 The kernels of session 2: coalesced or not?
- 4 Rules

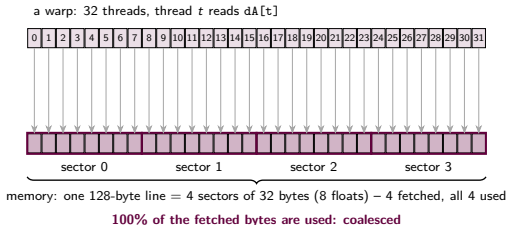
# Coalescing: a warp accesses memory as one

Coalescing is about how the 32 threads of a **warp** access global memory *together*.

- The threads of a warp execute each instruction **in lockstep** – a load or a store included
- So a warp's load is *one* request for 32 addresses; memory serves it in units of **32-byte sectors** (8 floats), grouped in 128-byte lines
- Example: thread  $t$  reads  $dA[t]$  – 32 consecutive floats, one 128-byte line:  
**4 sectors fetched, all 4 used**

**Definition.** A warp access is **coalesced** when *every byte of every sector it fetches is used* by the warp – the number of sectors fetched is the minimum for the bytes needed: 4 sectors for 32 floats.

```
__global__ void copyAligned(int n)
{
    int t = blockIdx.x * blockDim.x + threadIdx.x;
    if (t < n) { dB[t] = dA[t]; }
}
```

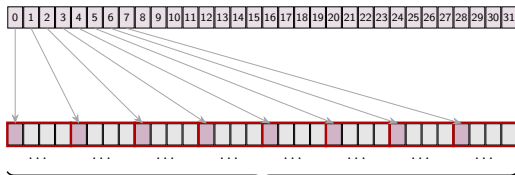


# Coalescing (cont.): what a scattered access costs

- If the 32 addresses are scattered, *every sector they touch* is fetched **in full** – memory does not deliver less than a sector
- Example: thread  $t$  reads  $dA[8 * t]$  – the 32 addresses fall in 32 different sectors: 1024 bytes moved for 128 bytes used, **8× the traffic**
- **Not coalesced**: 7/8 of every fetched sector is thrown away
- Global-memory bandwidth is the scarcest resource of the GPU; a kernel that wastes 7/8 of it cannot be fast, whatever else it does

```
__global__ void copyStride8(int n)
{
    int t = blockIdx.x * blockDim.x + threadIdx.x;
    if (t < n) { dB[t] = dA[8 * t]; }
}
```

a warp: 32 threads, thread  $t$  reads  $dA[t * \text{stride}]$ ,  $\text{stride} \geq 8$



memory: 32 different sectors fetched (8 shown), 1 float used in each

**1/8 of the fetched bytes are used: 8× the traffic of a coalesced access**

each sector is 32 bytes = 8 floats; grey = fetched but unused

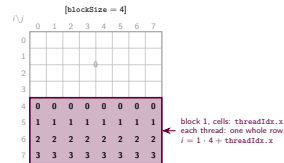
- 1 Recap of session 2
- 2 Coalesced memory access
- 3 The kernels of session 2: coalesced or not?**
- 4 Rules

# One thread per row

- Thread  $i$  multiplies the whole row  $A[i][\cdot]$ : a loop over  $j$
- At one step of the loop, the 32 threads of a warp have 32 **consecutive**  $i$  and the **same**  $j$
- Is the access to  $\text{dA}$  coalesced?

```
--global__ void multiplyArray2D(int n, float c)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        for (int j = 0; j < n; j++) { dA[i][j] *= c; }
    }
}

...
int blockSize = 1024;
int numBlocks = (N + blockSize - 1) / blockSize;
multiplyArray2D<<<numBlocks, blockSize>>>(N, c);
```

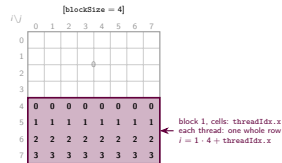


# One thread per row

- Thread  $i$  multiplies the whole row  $A[i][\cdot]$ : a loop over  $j$
- At one step of the loop, the 32 threads of a warp have 32 **consecutive  $i$**  and the **same  $j$**
- Is the access to  $\text{dA}$  coalesced?
- **No**:  $A[i][j]$  and  $A[i+1][j]$  are  $4N$  bytes apart – 32 sectors for 32 floats,  $1/8$  of the fetched bytes used
- And only  $N$  threads in total: too few to fill the GPU anyway

```
--global__ void multiplyArray2D(int n, float c)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        for (int j = 0; j < n; j++) { dA[i][j] *= c; }
    }

    ...
    int blockSize = 1024;
    int numBlocks = (N + blockSize - 1) / blockSize;
    multiplyArray2D<<<numBlocks, blockSize>>>(N, c);
}
```

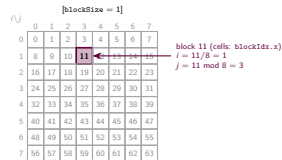


# One element per block/thread, $N^2$ blocks

- One thread per block: a warp has **one active thread** and 31 empty lanes
- Its single access is one float of one sector
- Is the access to dA coalesced?

```
--global__ void multiplyArray2D(int n, float c)
{
    int i = blockIdx.x / n;
    int j = blockIdx.x % n;
    dA[i][j] *= c;
}

...
multiplyArray2D<<<N * N, 1>>>(N, c);
```

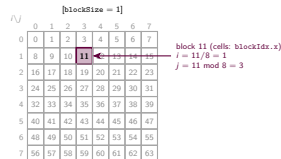


# One element per block/thread, $N^2$ blocks

- One thread per block: a warp has **one active thread** and 31 empty lanes
- Its single access is one float of one sector
- **Is the access to dA coalesced?**
- **No:** one sector fetched per useful float (1/8 of the bytes), and 31 of the 32 lanes idle – in this order of the blocks as in the other

```
__global__ void multiplyArray2D(int n, float c)
{
    int i = blockIdx.x / n;
    int j = blockIdx.x % n;
    dA[i][j] *= c;
}

...
multiplyArray2D<<<N * N, 1>>>(N, c);
```

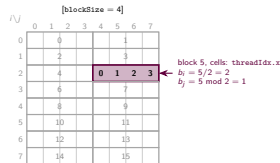


**(a) 1D grid, 1D blocks, threads row aligned**

- A block is a piece of a row:  
 $i = b_i, j = b_j$   
`blockDim.x + threadIdx.x`
- A warp = 32 consecutive  
`threadIdx.x = 32`  
**consecutive**  $j$ , same  $i$
- Is the access to dA  
**coalesced?**

```
__global__ void multiplyArray2D(int n, float c)
{
    int blocksPerRow = gridDim.x / n;
    int bi = blockIdx.x / blocksPerRow;
    int bj = blockIdx.x % blocksPerRow;
    int i = bi;
    int j = bj * blockDim.x + threadIdx.x;
    if (j < n) { dA[i][j] *= c; }
}

...
int blockSize = 1024;
int blocksPerRow =
    (N + blockSize - 1) / blockSize;
int nb = N * blocksPerRow;
multiplyArray2D<<<nb, blockSize>>>(N, c);
```



université  
PARIS-SACLAY

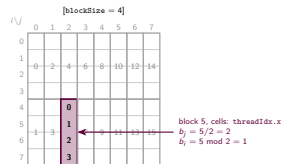
- 
- blockSize = 4
- $i \setminus j$
- |   |    |   |   |   |    |   |   |
|---|----|---|---|---|----|---|---|
| 0 | 1  | 2 | 3 | 4 | 5  | 6 | 7 |
| 0 | 0  |   |   |   | 1  |   |   |
| 1 | 2  |   |   |   | 3  |   |   |
| 2 | 4  |   |   |   | 0  | 1 | 2 |
| 3 | 6  |   |   |   | 7  |   |   |
| 4 | 8  |   |   |   | 9  |   |   |
| 5 | 10 |   |   |   | 11 |   |   |
| 6 | 12 |   |   |   | 13 |   |   |
| 7 | 14 |   |   |   | 15 |   |   |
- block 5, cells: threadIdx.x
- $b_i = 5/2 = 2$   
 $b_j = 5 \bmod 2 = 1$

## (b) 1D grid, 1D blocks, threads column aligned

- A block is a piece of a column:  $i = b_j$ .  
 $\text{blockDim.x} + \text{threadIdx.x}$ ,  
 $j = b_j$
- A warp = 32 **consecutive**  $i$ ,  
same  $j$
- Is the access to  $\text{dA}$   
coalesced?

```
--global__ void multiplyArray2D(int n, float c)
{
    int blocksPerCol = blockDim.x / n;
    int bj = blockIdx.x / blocksPerCol;
    int bi = blockIdx.x % blocksPerCol;
    int i = bi * blockDim.x + threadIdx.x;
    int j = bj;
    if (i < n) { dA[i][j] *= c; }
}

...
int blockSize = 1024;
int blocksPerCol =
    (N + blockSize - 1) / blockSize;
int nb = N * blocksPerCol;
multiplyArray2D<<<nb, blockSize>>>(N, c);
```

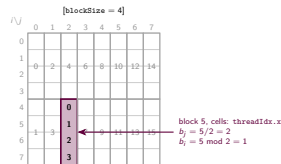


## (b) 1D grid, 1D blocks, threads column aligned

- A block is a piece of a column:  $i = b_j$ .  
blockDim.x + threadIdx.x,  
 $j = b_j$
- A warp = 32 **consecutive**  $i$ ,  
same  $j$
- Is the access to dA  
**coalesced?**
- **No:**  $A[i \dots i + 31][j]$  are  $4N$   
bytes apart – 32 sectors, one  
float used in each

```
__global__ void multiplyArray2D(int n, float c)
{
    int blocksPerCol = gridDim.x / n;
    int bj = blockIdx.x / blocksPerCol;
    int bi = blockIdx.x % blocksPerCol;
    int i = bi * blockDim.x + threadIdx.x;
    int j = bj;
    if (i < n) { dA[i][j] *= c; }
}

...
int blockSize = 1024;
int blocksPerCol =
    (N + blockSize - 1) / blockSize;
int nb = N * blocksPerCol;
multiplyArray2D<<<nb, blockSize>>>>(N, c);
```



université  
PARIS-SACLAY

- 
- [blockSize = 4]
- |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 0 |   |   |   |   |   |   |   |
| 1 |   |   |   |   |   |   |   |
| 2 |   |   |   |   |   |   |   |
| 3 |   |   |   |   |   |   |   |
| 4 |   |   |   |   |   |   |   |
| 5 |   |   |   |   |   |   |   |
| 6 |   |   |   |   |   |   |   |
| 7 |   |   |   |   |   |   |   |
- block 6, cells: threadIdx.x = 6/4 = 1  
by = 6 mod 4 = 2

université  
PARIS-SACLAY

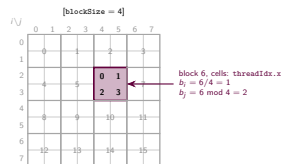
- 
- Diagram illustrating a 2D grid with 8 rows and 16 columns. The grid is divided into blocks of size 4x4. A specific block is highlighted in purple, containing the values 0, 2, 1, and 3. An arrow points from the text "block 6, cells: threadIdx.x = 6/4 = 1, by = 6 mod 4 = 2" to the highlighted block.

# (d) 1D grid, tile blocks, threads row aligned

- Same tiles;  
 $t_i = \text{threadIdx.x} / 32$ ,  
 $t_j = \text{threadIdx.x} \bmod 32$
- A warp = one **row** of the tile: 32 consecutive  $j$ , same  $i$
- Is the access to  $\text{dA}$  coalesced?

```
--global__ void multiplyArray2D(int n, float c)
{
    int side = sqrtf(blockDim.x);
    int tilesPerRow = (n + side - 1) / side;
    int bi = blockIdx.x / tilesPerRow;
    int bj = blockIdx.x % tilesPerRow;
    int ti = threadIdx.x / side;
    int tj = threadIdx.x % side;
    int i = bi * side + ti;
    int j = bj * side + tj;
    if (i < n && j < n) { dA[i][j] *= c; }
}

...
int blockSize = 1024;
int side = sqrtf(blockSize);
int tilesPerRow = (N + side - 1) / side;
int nb = tilesPerRow * tilesPerRow;
multiplyArray2D<<<nb, blockSize>>>(N, c);
```

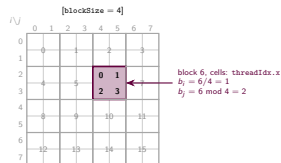


**(d) 1D grid, tile blocks, threads row aligned**

- Same tiles;  
 $t_i = \text{threadIdx.x}/32$ ,  
 $t_j = \text{threadIdx.x} \bmod 32$
- A warp = one **row** of the tile: 32 consecutive  $j$ , same  $i$
- **Is the access to  $dA$  coalesced?**
- **Yes:** 32 consecutive floats of a row of  $A$  – 4 sectors, all used, as in (a)

```
__global__ void multiplyArray2D(int n, float c)
{
    int side = sqrtf(blockDim.x);
    int tilesPerRow = (n + side - 1) / side;
    int bi = blockIdx.x / tilesPerRow;
    int bj = blockIdx.x % tilesPerRow;
    int ti = threadIdx.x / side;
    int tj = threadIdx.x % side;
    int i = bi * side + ti;
    int j = bj * side + tj;
    if (i < n && j < n) { dA[i][j] *= c; }
}

...
int blockSize = 1024;
int side = sqrtf(blockSize);
int tilesPerRow = (N + side - 1) / side;
int nb = tilesPerRow * tilesPerRow;
multiplyArray2D<<<(nb, blockSize>>>(N, c);
```

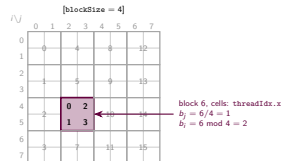


# (c) alternative: tiles numbered column by column, threads column aligned

- Only the **numbering of the blocks** changed: consecutive blocks walk down a column of tiles
- Inside a block nothing changed: a warp is still one column of the tile
- Is the access to **dA** coalesced?

```
--global__ void multiplyArray2D(int n, float c)
{
    int side = sqrtf(blockDim.x);
    int tilesPerCol = (n + side - 1) / side;
    int bj = blockIdx.x / tilesPerCol;
    int bi = blockIdx.x % tilesPerCol;
    int ti = threadIdx.x % side;
    int tj = threadIdx.x / side;
    int i = bi * side + ti;
    int j = bj * side + tj;
    if (i < n && j < n) { dA[i][j] *= c; }
}

...
int blockSize = 1024;
int side = sqrtf(blockSize);
int tilesPerCol = (N + side - 1) / side;
int nb = tilesPerCol * tilesPerCol;
multiplyArray2D<<<nb, blockSize>>>(N, c);
```

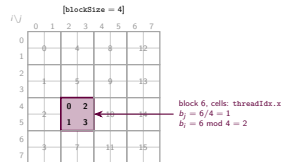


# (c) alternative: tiles numbered column by column, threads column aligned

- Only the **numbering of the blocks** changed: consecutive blocks walk down a column of tiles
- Inside a block nothing changed: a warp is still one column of the tile
- Is the access to dA coalesced?
- **No**: the verdict is decided *inside* the warp; how the blocks are numbered plays no role

```
--global__ void multiplyArray2D(int n, float c)
{
    int side = sqrtf(blockDim.x);
    int tilesPerCol = (n + side - 1) / side;
    int bj = blockIdx.x / tilesPerCol;
    int bi = blockIdx.x % tilesPerCol;
    int ti = threadIdx.x / side;
    int tj = threadIdx.x / side;
    int i = bi * side + ti;
    int j = bj * side + tj;
    if (i < n && j < n) { dA[i][j] *= c; }
}

...
int blockSize = 1024;
int side = sqrtf(blockSize);
int tilesPerCol = (N + side - 1) / side;
int nb = tilesPerCol * tilesPerCol;
multiplyArray2D<<<nb, blockSize>>>(N, c);
```

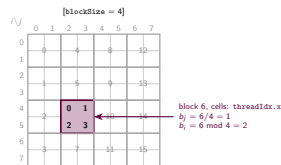


# (d) alternative: tiles numbered column by column, threads row aligned

- Blocks numbered column by column, threads row aligned: a warp is one row of the tile
- Is the access to dA coalesced?

```
__global__ void multiplyArray2D(int n, float c)
{
    int side = sqrtf(blockDim.x);
    int tilesPerCol = (n + side - 1) / side;
    int bj = blockIdx.x / tilesPerCol;
    int bi = blockIdx.x % tilesPerCol;
    int ti = threadIdx.x / side;
    int tj = threadIdx.x % side;
    int i = bi * side + ti;
    int j = bj * side + tj;
    if (i < n && j < n) { dA[i][j] *= c; }
}
```

```
...
int blockSize = 1024;
int side = sqrtf(blockSize);
int tilesPerCol = (N + side - 1) / side;
int nb = tilesPerCol * tilesPerCol;
multiplyArray2D<<<nb, blockSize>>>(N, c);
```



**(d) alternative: tiles numbered column by column, threads row aligned**

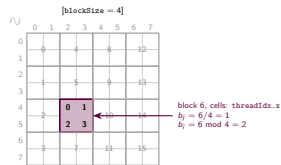
- Blocks numbered column by column, threads row aligned: a warp is one row of the tile
- **Is the access to dA coalesced?**
- **Yes:** as in (d) – 32 consecutive floats of a row

```

__global__ void multiplyArray2D(int n, float c)
{
    int side = sqrtf(blockDim.x);
    int tilesPerCol = (n + side - 1) / side;
    int bj = blockIdx.x / tilesPerCol;
    int bi = blockIdx.x % tilesPerCol;
    int ti = threadIdx.x / side;
    int tj = threadIdx.x % side;
    int i = bi * side + ti;
    int j = bj * side + tj;
    if (i < n && j < n) { dA[i][j] *= c; }
}

```

```
int blockSize = 1024;
int side = sqrtf(blockSize);
int tilesPerCol = (N + side - 1) / side;
int nb = tilesPerCol * tilesPerCol;
multiplyArray2D<<<nb, blockSize>>>(N, c);
```

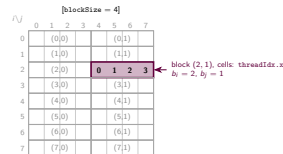


# (a) 2D grids/1D blocks, threads row aligned

- Case (a) with dim3:  
 $b_i = \text{blockIdx.x}$ ,  
 $b_j = \text{blockIdx.y}$ , no  
arithmetic
- A warp = 32 consecutive  $j$ ,  
same  $i$  – exactly as with 1D  
indices
- Is the access to  $\text{dA}$   
coalesced?

```
--global__ void multiplyArray2D(int n, float c)
{
    int i = blockIdx.x;
    int j = blockIdx.y * blockDim.x + threadIdx.x;
    if (j < n) { dA[i][j] *= c; }
}

...
int blockSize = 1024;
dim3 dimGrid;
dimGrid.x = N;
dimGrid.y = (N + blockSize - 1) / blockSize;
dimGrid.z = 1;
multiplyArray2D<<<dimGrid, blockSize>>>(N, c);
```

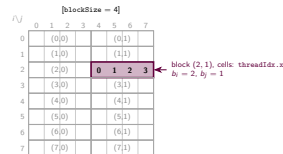


# (a) 2D grids/1D blocks, threads row aligned

- Case (a) with dim3:  
 $b_i = \text{blockIdx.x}$ ,  
 $b_j = \text{blockIdx.y}$ , no arithmetic
- A warp = 32 consecutive  $j$ , same  $i$  – exactly as with 1D indices
- Is the access to  $\text{dA}$  coalesced?
- Yes: the indices changed, the mapping did not – 4 sectors, all used

```
--global__ void multiplyArray2D(int n, float c)
{
    int i = blockIdx.x;
    int j = blockIdx.y * blockDim.x + threadIdx.x;
    if (j < n) { dA[i][j] *= c; }
}

...
int blockSize = 1024;
dim3 dimGrid;
dimGrid.x = N;
dimGrid.y = (N + blockSize - 1) / blockSize;
dimGrid.z = 1;
multiplyArray2D<<<dimGrid, blockSize>>>(N, c);
```

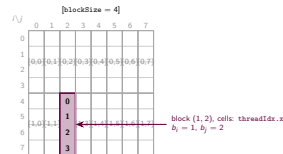


# (b) 2D grids/1D blocks, threads column aligned

- Case (b) with dim3: a warp = 32 consecutive  $i$ , same  $j$
- Is the access to  $dA$  coalesced?

```
__global__ void multiplyArray2D(int n, float c)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y;
    if (i < n) { dA[i][j] *= c; }
}

...
int blockSize = 1024;
dim3 dimGrid;
dimGrid.x = (N + blockSize - 1) / blockSize;
dimGrid.y = N;
dimGrid.z = 1;
multiplyArray2D<<<dimGrid, blockSize>>>(N, c);
```

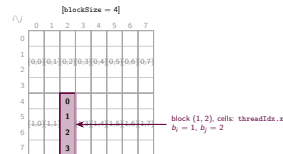


## (b) 2D grids/1D blocks, threads column aligned

- Case (b) with dim3: a warp = 32 consecutive  $i$ , same  $j$
- Is the access to  $\mathbf{dA}$  coalesced?
- **No**: 32 sectors, one float used in each

```
__global__ void multiplyArray2D(int n, float c)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y;
    if (i < n) { dA[i][j] *= c; }
}

...
int blockSize = 1024;
dim3 dimGrid;
dimGrid.x = (N + blockSize - 1) / blockSize;
dimGrid.y = N;
dimGrid.z = 1;
multiplyArray2D<<<dimGrid, blockSize>>>(N, c);
```

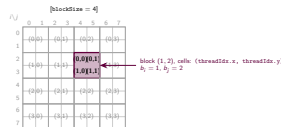


# (c) 2D grids/blocks, threads column aligned

- $32 \times 32$  blocks;  
 $t_i = \text{threadIdx.x}$ ,  
 $t_j = \text{threadIdx.y}$
- Warps: 32 consecutive  
 $\text{threadIdx.x}$ , same  
 $\text{threadIdx.y} = \text{one column}$   
of the tile
- Is the access to  $\text{dA}$   
coalesced?

```
__global__ void multiplyArray2D(int n, float c)
{
    int bi = blockIdx.x;
    int bj = blockIdx.y;
    int ti = threadIdx.x;
    int tj = threadIdx.y;
    int i = bi * blockDim.x + ti;
    int j = bj * blockDim.y + tj;
    if (i < n && j < n) { dA[i][j] *= c; }

    ...
    dim3 dimBlock;
    dimBlock.x = 32;
    dimBlock.y = 32;
    dimBlock.z = 1;
    dim3 dimGrid;
    dimGrid.x = (N + dimBlock.x - 1) / dimBlock.x;
    dimGrid.y = (N + dimBlock.y - 1) / dimBlock.y;
    dimGrid.z = 1;
    multiplyArray2D<<<dimGrid, dimBlock>>>(N, c);
}
```

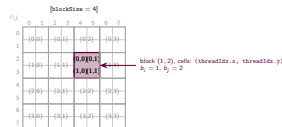


# (c) 2D grids/blocks, threads column aligned

- $32 \times 32$  blocks;  
 $t_i = \text{threadIdx.x}$ ,  
 $t_j = \text{threadIdx.y}$
- Warps: 32 consecutive  
 $\text{threadIdx.x}$ , same  
 $\text{threadIdx.y} = \text{one column}$   
 of the tile
- Is the access to  $\text{dA}$   
 coalesced?
- No:  $\text{threadIdx.x}$  on the  
 row index  $i$  sends the warp  
 down a column – 32 sectors

```
__global__ void multiplyArray2D(int n, float c)
{
    int bi = blockIdx.x;
    int bj = blockIdx.y;
    int ti = threadIdx.x;
    int tj = threadIdx.y;
    int i = bi * blockDim.x + ti;
    int j = bj * blockDim.y + tj;
    if (i < n && j < n) { dA[i][j] *= c; }

    ...
    dim3 dimBlock;
    dimBlock.x = 32;
    dimBlock.y = 32;
    dimBlock.z = 1;
    dim3 dimGrid;
    dimGrid.x = (N + dimBlock.x - 1) / dimBlock.x;
    dimGrid.y = (N + dimBlock.y - 1) / dimBlock.y;
    dimGrid.z = 1;
    multiplyArray2D<<<dimGrid, dimBlock>>>(N, c);
}
```

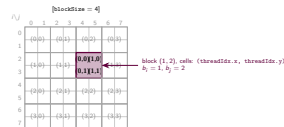


# (d) 2D grids/blocks, threads row aligned

- $t_i = \text{threadIdx.y}$ ,  
 $t_j = \text{threadIdx.x}$ : a warp  
= one **row** of the tile
- Is the access to **data**  
coalesced?

```
__global__ void multiplyArray2D(int n, float c)
{
    int bi = blockIdx.x;
    int bj = blockIdx.y;
    int ti = threadIdx.y;
    int tj = threadIdx.x;
    int i = bi * blockDim.y + ti;
    int j = bj * blockDim.x + tj;
    if (i < n && j < n) { dA[i][j] *= c; }
}

...
dim3 dimBlock;
dimBlock.x = 32;
dimBlock.y = 32;
dimBlock.z = 1;
dim3 dimGrid;
dimGrid.x = (N + dimBlock.x - 1) / dimBlock.x;
dimGrid.y = (N + dimBlock.y - 1) / dimBlock.y;
dimGrid.z = 1;
multiplyArray2D<<<dimGrid, dimBlock>>>(N, c);
```

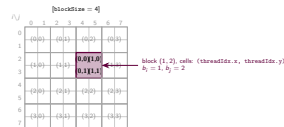


# (d) 2D grids/blocks, threads row aligned

- $t_i = \text{threadIdx.y}$ ,  
 $t_j = \text{threadIdx.x}$ : a warp  
= one **row** of the tile
- Is the access to **data**  
**coalesced**?
- **Yes**:  $\text{threadIdx.x}$  on the  
column index  $j - 32$   
consecutive floats, 4 sectors

```
__global__ void multiplyArray2D(int n, float c)
{
    int bi = blockIdx.x;
    int bj = blockIdx.y;
    int ti = threadIdx.y;
    int tj = threadIdx.x;
    int i = bi * blockDim.y + ti;
    int j = bj * blockDim.x + tj;
    if (i < n && j < n) { dA[i][j] *= c; }
}

...
dim3 dimBlock;
dimBlock.x = 32;
dimBlock.y = 32;
dimBlock.z = 1;
dim3 dimGrid;
dimGrid.x = (N + dimBlock.x - 1) / dimBlock.x;
dimGrid.y = (N + dimBlock.y - 1) / dimBlock.y;
dimGrid.z = 1;
multiplyArray2D<<<dimGrid, dimBlock>>>(N, c);
```

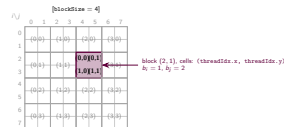


# (c) alternative: 2D grids (transposed)/blocks, threads column aligned

- $b_i = \text{blockIdx.y}$ ,  
 $b_j = \text{blockIdx.x}$ : the grid  
is transposed, the threads  
are as in (c)
- Is the access to dA  
coalesced?

```
--global__ void multiplyArray2D(int n, float c)
{
    int bi = blockIdx.y;
    int bj = blockIdx.x;
    int ti = threadIdx.x;
    int tj = threadIdx.y;
    int i = bi * blockDim.x + ti;
    int j = bj * blockDim.y + tj;
    if (i < n && j < n) { dA[i][j] *= c; }
}

...
dim3 dimBlock;
dimBlock.x = 32;
dimBlock.y = 32;
dimBlock.z = 1;
dim3 dimGrid;
dimGrid.x = (N + dimBlock.x - 1) / dimBlock.x;
dimGrid.y = (N + dimBlock.y - 1) / dimBlock.y;
dimGrid.z = 1;
multiplyArray2D<<<dimGrid, dimBlock>>>(N, c);
```

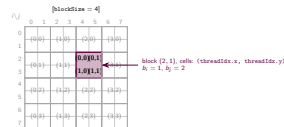


# (c) alternative: 2D grids (transposed)/blocks, threads column aligned

- $b_i = \text{blockIdx.y}$ ,  
 $b_j = \text{blockIdx.x}$ : the grid  
is transposed, the threads  
are as in (c)
- Is the access to dA  
coalesced?
- **No**: a warp is still a column  
of the tile; the block indices  
do not matter

```
__global__ void multiplyArray2D(int n, float c)
{
    int bi = blockIdx.y;
    int bj = blockIdx.x;
    int ti = threadIdx.x;
    int tj = threadIdx.y;
    int i = bi * blockDim.x + ti;
    int j = bj * blockDim.y + tj;
    if (i < n && j < n) { dA[i][j] *= c; }
}

...
dim3 dimBlock;
dimBlock.x = 32;
dimBlock.y = 32;
dimBlock.z = 1;
dim3 dimGrid;
dimGrid.x = (N + dimBlock.x - 1) / dimBlock.x;
dimGrid.y = (N + dimBlock.y - 1) / dimBlock.y;
dimGrid.z = 1;
multiplyArray2D<<<dimGrid, dimBlock>>>(N, c);
```

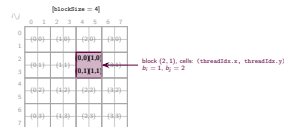


# (d) alternative: 2D grids (transposed)/blocks, threads row aligned

- Grid transposed, threads as in (d): a warp is a row of the tile
- Is the access to `dA` coalesced?

```
__global__ void multiplyArray2D(int n, float c)
{
    int bi = blockIdx.y;
    int bj = blockIdx.x;
    int ti = threadIdx.y;
    int tj = threadIdx.x;
    int i = bi * blockDim.y + ti;
    int j = bj * blockDim.x + tj;
    if (i < n && j < n) { dA[i][j] *= c; }
}

...
dim3 dimBlock;
dimBlock.x = 32;
dimBlock.y = 32;
dimBlock.z = 1;
dim3 dimGrid;
dimGrid.x = (N + dimBlock.x - 1) / dimBlock.x;
dimGrid.y = (N + dimBlock.y - 1) / dimBlock.y;
dimGrid.z = 1;
multiplyArray2D<<<dimGrid, dimBlock>>>(N, c);
```

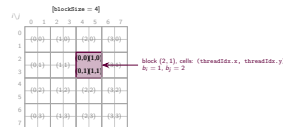


# (d) alternative: 2D grids (transposed)/blocks, threads row aligned

- Grid transposed, threads as in (d): a warp is a row of the tile
- Is the access to **data** coalesced?
- Yes: as in (d)

```
__global__ void multiplyArray2D(int n, float c)
{
    int bi = blockIdx.y;
    int bj = blockIdx.x;
    int ti = threadIdx.y;
    int tj = threadIdx.x;
    int i = bi * blockDim.y + ti;
    int j = bj * blockDim.x + tj;
    if (i < n && j < n) { dA[i][j] *= c; }
}

...
dim3 dimBlock;
dimBlock.x = 32;
dimBlock.y = 32;
dimBlock.z = 1;
dim3 dimGrid;
dimGrid.x = (N + dimBlock.x - 1) / dimBlock.x;
dimGrid.y = (N + dimBlock.y - 1) / dimBlock.y;
dimGrid.z = 1;
multiplyArray2D<<<dimGrid, dimBlock>>>(N, c);
```



# The answer: which ones are coalesced

| case                                  | a warp touches               | coalesced? |
|---------------------------------------|------------------------------|------------|
| one thread per row ( $N$ threads)     | 32 floats of a <b>column</b> | <b>no</b>  |
| one thread per block, $N^2$ blocks    | 1 float, 31 lanes idle       | <b>no</b>  |
| (a) 1D blocks, threads row aligned    | 32 floats of a <b>row</b>    | <b>yes</b> |
| (b) 1D blocks, threads column aligned | 32 floats of a <b>column</b> | <b>no</b>  |
| (c) tiles, threads column aligned     | 32 floats of a <b>column</b> | <b>no</b>  |
| (d) tiles, threads row aligned        | 32 floats of a <b>row</b>    | <b>yes</b> |

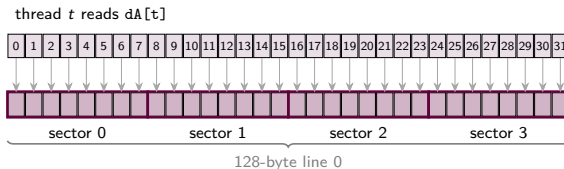
- 1D or 2D indices, block numbering, alternatives: **same verdict** – only the mapping of `threadIdx.x` counts
- Same arithmetic in all of them; the coalesced ones move  $8\times$  fewer sectors

- 1 Recap of session 2
- 2 Coalesced memory access
- 3 The kernels of session 2: coalesced or not?
- 4 Rules**

# Rule 1: contiguous and aligned – the ideal case

- Thread  $t$  reads  $dA[t]$ : warp  $w$  reads floats  $32w \dots 32w + 31$
- These 128 bytes start on a 128-byte boundary: exactly one line, **4 sectors**, every byte used
- The write  $dB[t]$  is the same pattern.  
**Rule:** 32 consecutive  $threadIdx.x \rightarrow$  32 consecutive, aligned floats – the pattern to aim for

```
__global__ void copyAligned(int n)
{
    int t = blockIdx.x * blockDim.x + threadIdx.x;
    if (t < n) { dB[t] = dA[t]; }
}
```

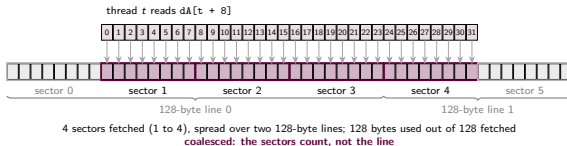


4 sectors fetched, 128 bytes used out of 128 fetched  
**coalesced: every byte fetched is used**

# Rule 2: contiguous, but not on a line boundary

- Thread  $t$  reads  $dA[t + 8]$ : warp  $w$  reads floats  $32w + 8 \dots 32w + 39$
- The 128 bytes start on a **sector** boundary but not on a line boundary: sectors 1 to 4, spread over two lines
- Still **4 sectors**, every byte used: **coalesced**
- Rule:** what is counted is the number of *sectors* fetched; the 128-byte line is not the unit

```
--global__ void copySectorOffset(int n)
{
    int t = blockIdx.x * blockDim.x + threadIdx.x;
    if (t < n) { dB[t] = dA[t + 8]; }
}
```



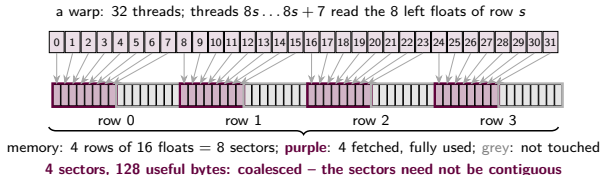
### Rule 3: the sectors need not be contiguous either

- A has 16 columns (2 sectors per row); thread  $t$  reads one element of the **left half** of row  $t/8$
- 8 consecutive threads read 8 consecutive floats, then the next 8 **jump** to the next row: 4 groups of 8 floats, 64 bytes apart
- **Is the access to dA coalesced?**

```
--global__ void leftHalves(int n)
{
    int t = blockIdx.x * blockDim.x + threadIdx.x;
    int i = t / (M / 2);
    int j = t % (M / 2);
    if (i < n) { dB[t] = dA[i][j]; }
}

...

int nb = (N * M / 2 + 255) / 256;
leftHalves<<<nb, 256>>>(N);
```

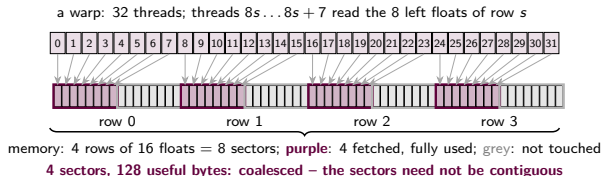


# Rule 3: the sectors need not be contiguous either

- A has 16 columns (2 sectors per row); thread  $t$  reads one element of the **left half** of row  $t/8$
- 8 consecutive threads read 8 consecutive floats, then the next 8 **jump** to the next row: 4 groups of 8 floats, 64 bytes apart
- **Is the access to dA coalesced?**
- **Yes:** 4 sectors fetched, every byte used – the traffic of 32 contiguous floats
- The unit is the **32-byte sector**, not the 128-byte line

```
--global__ void leftHalves(int n)
{
    int t = blockIdx.x * blockDim.x + threadIdx.x;
    int i = t / (M / 2);
    int j = t % (M / 2);
    if (i < n) { dB[t] = dA[i][j]; }
}

...
int nb = (N * M / 2 + 255) / 256;
leftHalves<<<nb, 256>>>(N);
```

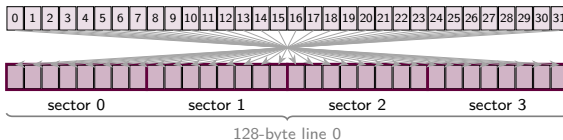


# Rule 4: any permutation within the sectors is free

- Thread  $t$  of warp  $w$  reads  $dA[32w + 31 - 1]$ : the same 32 floats as  $dA[t]$ , in **reverse order**
- One request of 32 addresses: the memory system sees the same **4 sectors**, whichever thread asks for which float
- Coalesced. Rule:** coalescing is a property of the set of addresses of a warp, not of their order

```
__global__ void copyReversed(int n)
{
    int t = blockIdx.x * blockDim.x + threadIdx.x;
    int w = t / 32, l = t % 32;
    if (t < n) { dB[t] = dA[w * 32 + (31 - l)]; }
}
```

thread  $t$  reads  $dA[31 - t]$  (reversed order within the warp)

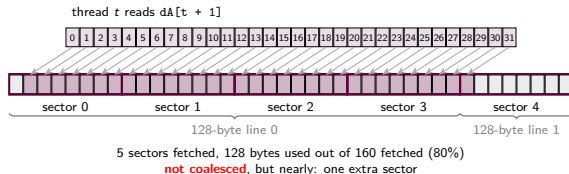


the same 4 sectors as  $dA[t]$ ; 128 bytes used out of 128 fetched  
**coalesced: the order of the threads within the sectors does not matter**

# Rule 5: contiguous but misaligned – one extra sector

- Thread  $t$  reads  $dA[t + 1]$ : 32 consecutive floats, starting one float after a sector boundary
- They straddle **5 sectors**: 160 bytes fetched for 128 used (80%)
- Not coalesced by the definition, but the cost is small: 5/4 of the traffic, not 8×
- Rule:** align the start of the data of a warp on a sector (cudaMalloc returns aligned addresses; the offset comes from the index formula)

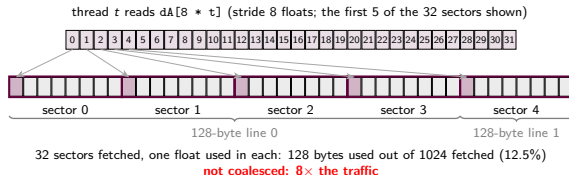
```
__global__ void copyMisaligned(int n)
{
    int t = blockIdx.x * blockDim.x + threadIdx.x;
    if (t < n) { dB[t] = dA[t + 1]; }
}
```



# Rule 6: a stride between the threads

- Thread  $t$  reads  $dA[8 * t]$ : consecutive threads are 8 floats = one sector apart
- Each thread is **alone in its sector**: 32 sectors fetched, 1 float used in each – 1024 bytes moved for 128 used
- A stride of  $s$  floats touches  $\min(32, 4s)$  sectors:  $s = 2 \rightarrow 8$  (50% used),  $4 \rightarrow 16$  (25%),  $\geq 8 \rightarrow 32$  (12.5%)
- **Rule:** a stride  $\geq 8$  floats is the worst case – a column access in row-major storage has  $s = N$

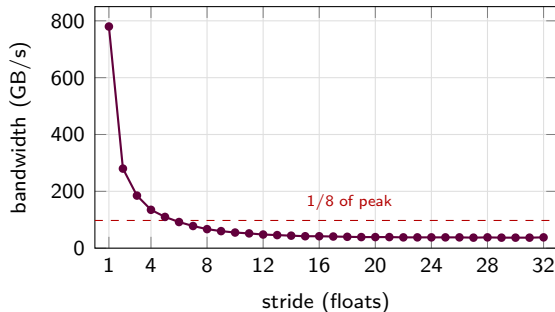
```
__global__ void copyStride8(int n)
{
    int t = blockIdx.x * blockDim.x + threadIdx.x;
    if (t < n) { dB[t] = dA[8 * t]; }
}
```



# The stride case, measured

- $dB[t] = dA[t * \text{stride}]$ , stride = 1...32  
(src/coalescing-stride.cu): the bandwidth collapses past stride 1 and settles at  $\approx 1/20$  of the peak, **below the 1/8** the sector count predicts. Why?
- **Transactions, not only bytes**: a warp load touching 32 sectors is 32 memory transactions (4 when coalesced); the L1/L2 pipeline handles a fixed number per cycle and saturates on the *count* of sectors before the byte rate
- **DRAM granularity**: on most GPUs DRAM is read in 64-byte bursts (2 sectors); a lone 32-byte sector costs a full burst, so DRAM traffic is  $16\times$ , not  $8\times$  – and the 32 bursts hit 32 different DRAM pages
- Curve after M. Harris, *How to Access Global Memory Efficiently*, NVIDIA blog

Copy with stride on a Tesla V100



# The rules, in one slide

- A warp's load or store is **one request for 32 addresses**; memory serves it in **32-byte sectors** (8 floats)
- **Coalesced** = every byte of every fetched sector is used: 4 sectors for 32 floats. What is counted is the number of sectors, not the number of lines
- Contiguous and aligned (rule 1), contiguous from any sector boundary (rule 2), 4 full sectors anywhere (rule 3), any permutation inside them (rule 4): all **coalesced**
- Misaligned by a few floats (rule 5): one extra sector, **minor**. Stride of  $s$  floats (rule 6):  $\min(32, 4s)$  sectors, **up to 8× the traffic** – and in practice worse
- For a 2D array: a warp must read a **contiguous piece of the storage order** – a piece of a row in row-major, of a column in column-major. The mapping of `threadIdx.x` decides; 1D or 2D blocks and the block numbering do not

## Contact

Oguz Kaya

Université Paris-Saclay and LISN, Gif-sur-Yvette, France

[oguz.kaya@universite-paris-saclay.fr](mailto:oguz.kaya@universite-paris-saclay.fr)

[www.oguzkaya.com](http://www.oguzkaya.com)

Course material: [github.com/kayaogz/2627-m2dipac-gpuprogramming](https://github.com/kayaogz/2627-m2dipac-gpuprogramming)